

ClipMind End-to-End Deep Dive

Purpose of This Document

This document is written to help a student explain the project end to end with confidence. It is intentionally detailed. The goal is not only to describe what the project does, but also to explain how the system is structured, why major technical decisions were taken, where the difficult parts are, what bugs occurred, and how the final workflow actually operates.

If an examiner asks, "Did you really understand this project?" this document should give you enough depth to answer architecture questions, feature questions, debugging questions, and novelty questions.

One-Line Definition of the Project

ClipMind is an AI-assisted video editor in which the transcript is the control layer for editing, captions, and B-roll, while a visible timeline remains the final surface for manual correction and finishing.

What Problem the Project Solves

Traditional video editing is slow because the user must search visually and aurally through footage. This becomes painful for:

- podcasts
- talking-head videos
- interviews
- lyric videos
- multilingual creator content

This project reduces that friction by converting speech and lyrics into structured text with timestamps, then using that text to drive editing operations.

The project also addresses another practical problem: many tools handle English reasonably well but perform poorly on Indian-language subtitles, non-English B-roll retrieval, and mixed-language creator content. The project attempts to bridge that gap by adding language-aware logic into the transcription, caption, and B-roll pipeline.

High-Level System View

The project has two main halves.

1. Frontend Editor

The frontend is a React + TypeScript application. It handles:

- the editor layout
- project/media selection
- transcript panel
- caption style UI
- B-roll Studio
- preview player
- timeline rendering
- manual adjustment controls

The frontend is not doing the actual heavy media processing. It acts as the orchestration and interaction surface.

2. Backend Processing Engine

The backend is a FastAPI application. It handles:

- project persistence
- media and transcript storage
- timeline state management
- AI transcription calls
- B-roll planning and candidate generation
- subtitle file generation
- preview/export rendering
- background jobs

The backend is where the real decision logic and media operations live.

Main Architectural Principle

The central principle of this system is:

text and timeline are linked, but neither replaces the other

This is important. The project is not just a transcript viewer and not just a timeline editor.

- The transcript provides semantic control.
- The timeline provides visual/manual control.
- AI helps create draft decisions.
- The user can still refine, reject, move, trim, or restyle the result.

That human-in-the-loop design is one of the strongest academic justifications of the project.

Core Backend Modules

backend/app/main.py

This file boots the FastAPI app, initializes the database, mounts static directories, and starts background workers for ingest and rendering.

Important idea:

- the app is not only request-response
- it also has background job processing
- preview and export do not need to block the UI

backend/app/models.py

This file defines the persistent entities:

- project
- timeline
- timeline versions
- media assets
- transcripts
- B-roll slots
- B-roll candidates
- B-roll choices
- operation history
- background jobs

This is important to understand during a viva because it shows the project is not a toy in-memory demo. It keeps state across the main editing workflow.

backend/app/transcription_service.py

This is one of the most important files in the entire project. It contains:

- transcription provider routing
- language normalization
- low-coverage detection
- retry logic
- language guard retry
- gap rescue logic

- Indic-language routing/probing
- transcript quality heuristics

This file answers the question:

"How does the system decide what transcript to trust?"

backend/app/timeline_service.py

This file is the edit engine for the timeline. It applies operations such as:

- add clip
- trim clip
- split clip
- merge clips
- add subtitles
- move captions
- add/move/trim/delete B-roll clips
- set B-roll opacity
- set volume/speed
- ripple edit
- move clips

This is effectively the state machine for the editor.

backend/app/render_service.py

This file is the render path. It:

- builds caption overlays
- creates ASS subtitle files
- composes B-roll overlays
- configures FFmpeg filter graphs
- generates preview and export outputs

This module became especially important when fixing Kannada/Indic subtitle shaping issues.

backend/app/routers/transcript.py

This route file handles transcript generation and transcript editing APIs.

backend/app/routers/vibe.py

This route file handles one-click higher-level editing actions such as:

- auto cut pauses
- trim start/end
- add captions

It is basically the "AI action launcher" layer.

backend/app/routers/broll.py

This file controls B-roll planning and slot workflows:

- plan beats
- suggest candidates
- async generation
- choose/reroll/reject candidates
- sync chosen clips to timeline
- undo the last B-roll sync

Core Frontend Modules

frontend/src/App.tsx

This is the main editor container and orchestration layer.

It is responsible for:

- editor state
- transcript language selection
- transcript rendering and selection
- preview state
- caption style selection
- B-roll Studio state
- timeline selections
- job polling
- syncing backend results into the UI

This file is large because it acts as the interaction hub.

frontend/src/components/Timeline.tsx

This is the visual timeline engine.

It handles:

- time ruler
- waveform bars
- video and audio lanes
- caption blocks
- B-roll blocks
- dragging, trimming, selection
- snapping behavior

This component is what makes the editor feel like an editor instead of just an AI dashboard.

frontend/src/lib/api.ts

This file centralizes all HTTP communication with the backend. It is important because it keeps API interactions explicit and typed.

frontend/src/types.ts

This defines the shared frontend data model for:

- clips
- tracks
- transcripts
- B-roll slots and candidates
- jobs
- operation history

End-to-End Workflow

Step 1: Project Creation

The user creates a project. A project has default FPS and canvas dimensions. The frontend calls the project API. The backend creates the project record and initializes timeline state.

Step 2: Media Upload or URL Ingest

The user uploads a local video or requests URL ingest. The backend stores the media file and metadata, including duration. The frontend refreshes the media list and chooses the active source clip.

Step 3: Transcript Generation

This is the first major AI stage.

The frontend sends:

- project id
- asset id
- source language or auto
- optional prompt

The backend then:

1. resolves the media file path
2. computes or verifies duration
3. checks whether a recent reusable transcript already exists
4. if not, calls `generate_transcript(...)`

Inside `generate_transcript(...)` the system may:

- use Groq Whisper as primary transcription path
- retry with a different Whisper model
- run gap rescue for sparse output
- apply language heuristics
- route or probe Sarvam for Indic cases
- save the chosen transcript

Finally the transcript is stored and the video is placed on the timeline.

Step 4: Transcript-Driven Editing

Once transcript words exist, the frontend can:

- select words
- select ranges
- delete filler sections
- edit individual words
- cut the timeline based on transcript

That is the key project idea: text is no longer passive output. It becomes an editing interface.

Step 5: Vibe Actions

The project includes higher-level actions such as:

- auto cut pauses
- trim start/end
- add captions

These actions are not direct FFmpeg commands from the UI. They are routed through structured backend logic, which updates timeline state and often queues a preview render.

Step 6: Caption Generation

When the user adds captions, the backend:

- reads transcript words
- groups them into overlays
- applies style settings
- stores them as text overlays on the timeline

Then the render path uses ASS subtitle generation for actual burned-in preview/export behavior.

Step 7: B-roll Planning

The B-roll pipeline does not directly search from raw transcript alone.

Instead, the system:

1. segments transcript into beats
2. creates concept text and visual intent
3. translates non-English meaning into English visual gloss when needed
4. expands search queries
5. generates or retrieves B-roll candidates
6. stores candidates per slot

The UI then lets the user inspect:

- original lyric or transcript line
- English meaning used for B-roll
- final query list

This is very important because it turns B-roll from a hidden AI guess into a controllable workflow.

Step 8: Timeline Editing

Once B-roll is chosen and synced, it becomes part of the overlay track. The timeline UI lets the user:

- move the B-roll block
- trim its duration
- adjust opacity
- reroll or delete it

That is how the project combines AI proposal with manual polish.

Step 9: Preview and Export

Rendering is job-based.

The frontend queues preview/export.

The backend:

- logs job state
- builds FFmpeg graph
- uses subtitle overlays and B-roll overlays
- outputs a file
- exposes job progress for the frontend

This allows the UI to remain responsive while media rendering runs in the background.

How Timeline State Works

Timeline state is represented as structured JSON stored in the database. This is a strong design choice.

Why this matters:

- operations can be replayed conceptually
- undo/redo is easier
- the frontend can receive a full updated timeline after each operation
- the system avoids ad hoc state mutations scattered everywhere

The operation pattern also makes the project easier to explain in academic terms because it shows deterministic editing logic rather than opaque side effects.

Why Transcript-Driven Editing Is a Strong Idea

Transcript-driven editing is useful because:

- users think in semantic units, not frame numbers
- speech videos are easier to edit by meaning than by waveform alone
- captioning and B-roll become downstream functions of the same timed text

This means one transcript becomes the common data layer for:

- cuts
- subtitles
- summary-like review
- B-roll beat generation

- timeline search and navigation

That is one of the best conceptual strengths of the project.

Detailed Explanation of the Transcription System

What the transcription layer is doing

The transcription layer is not only calling one model and trusting the answer. It uses multiple heuristics to decide whether the transcript is good enough.

Key checks include:

- coverage over duration
- suspicious long gaps
- low confidence or weak quality
- script mismatch for chosen language
- whether a retry candidate is better than the current result

Why this matters

If the system simply accepted the first transcript, many later stages would fail:

- captions would be wrong
- B-roll would become generic
- transcript editing would be meaningless

So transcription quality is upstream of almost everything.

Mixed-Language Improvement

One of the later improvements was to make auto mode more robust for mixed English + Indic content. The system now checks whether the resulting transcript text itself contains Indic script and, if so, probes the Indic route instead of trusting a purely English auto detection.

This is not full per-word language tagging yet, but it is a meaningful improvement to the auto pipeline.

Detailed Explanation of the Caption System

The initial problem

Complex scripts such as Kannada need proper text shaping. Some subtitle and font styling strategies that work for English can break ligatures and combining marks in Indic scripts.

What was wrong

The dangerous combination was:

- Latin-oriented font assumptions
- synthetic bold behavior
- inline karaoke-style color tags

These can split glyph clusters or make letters look visually broken.

The fix

The system now uses a safer ASS pipeline for complex scripts:

- explicit font fallback for Indic text
- disabled synthetic bold for fallback Indic fonts
- disabled inline karaoke highlight tags for Indic lines
- used safer whole-line visual styling instead of risky per-word styling

This preserved readability while still keeping preset identity where possible.

Why this is a good answer in a viva

If asked "What difficult engineering issue did you solve?", this is one of the best examples because it involves:

- Unicode shaping
- font fallback
- subtitle rendering
- tradeoffs between design style and text correctness

Detailed Explanation of the B-roll System

The first version of the problem

Directly using non-English transcript text as stock search queries gave weak results. For Kannada song lyrics, the system either searched poorly or collapsed into generic visuals.

The key insight

B-roll search works better from visual meaning than from raw lyric text.

So the pipeline was changed to:

- keep original transcript for captions
- convert non-English beat meaning into English visual gloss
- expand that into stock-friendly search queries

Why the debug UI matters

The project now exposes:

- original lyric
- English meaning used for B-roll
- final search queries

This is important academically because it turns AI from a black box into an inspectable decision system.

Human-in-the-loop value

The user can manually edit the English gloss and regenerate B-roll from that custom meaning. This is a strong design decision because it acknowledges that fully automatic semantic visual retrieval is imperfect, especially for songs and poetry.

Timeline V1 Design and Why It Matters

Timeline V1 is not just decoration. It is central to the product identity.

Without it, the system would look like a set of AI buttons. With it, the project becomes an editor.

The timeline includes:

- video lane
- audio lane
- caption lane
- waveform lane
- B-roll lane
- transcript assist lane

It supports:

- move
- trim
- split
- delete
- snap behavior
- lane mute/solo/lock

The B-roll lane was improved so it shows thumbnail-style blocks rather than tiny generic bars, which makes the interaction closer to a real editing experience.

Preview and Export Rendering

Why job-based rendering is used

Rendering can take time. If the backend tried to return preview/export synchronously, the UI would block and the experience would feel fragile.

Using jobs allows:

- progress tracking
- background rendering
- safer retry behavior
- better user feedback

Preview vs export

Preview is lower-cost and designed for iteration. Export is higher quality and meant for final output.

That distinction is important in video systems because users need fast feedback loops before requesting final render quality.

Important Bugs and What They Teach

Bug 1: Kannada captions rendered with broken glyphs

Root cause:

- complex text shaping broken by styling assumptions

Lesson:

- multilingual text rendering is not only a model problem; it is also a media rendering problem

Bug 2: "Original lyric" showed English gloss instead of original transcript

Root cause:

- the UI trusted B-roll reasoning payload instead of deriving transcript text directly from timed words

Lesson:

- debug metadata should not overwrite source truth

Bug 3: white captions appeared yellow for Indic text

Root cause:

- a style fix for color-led presets promoted highlight color too aggressively

Lesson:

- style fallback logic must be selective, not global

Bug 4: B-roll looked generic for non-English songs

Root cause:

- raw non-English lyric text was not a good stock search representation

Lesson:

- meaning conversion is often necessary before retrieval

Bug 5: Timeline looked too prototype-like and B-roll drag was unclear

Root cause:

- visual affordance and interaction area were too weak

Lesson:

- editor quality is not only about backend capability; interaction clarity matters

Why the Project Is More Than "We Used a Model"

An examiner may ask whether the project is just API glue. The correct answer is no.

The significant engineering work is in:

- orchestration
- data modeling
- timeline operations
- language-aware caption rendering
- retry and rescue logic for transcription
- B-roll semantic planning
- human-in-the-loop debugging and override surfaces
- preview/export pipeline

The base models are not the entire project. The value comes from how they are integrated, constrained, corrected, and presented to the user.

Strong Points to Say in a Viva

If asked "What is your main contribution?"

Say:

"The contribution is an integrated transcript-driven editor where text, timeline, captions, and B-roll are connected in one workflow, with language-aware handling for multilingual creator content."

If asked "What is technically challenging here?"

Say:

"The hardest parts are not only transcription. They include keeping timed transcript edits synchronized with timeline state, handling subtitle rendering correctly for Indic scripts, and translating non-English content into usable visual B-roll queries."

If asked "Why is this different from existing tools?"

Say:

"Many tools offer transcription or captions or AI B-roll separately. This project integrates them into one editable timeline-driven workflow and exposes intermediate AI reasoning for correction."

If asked "What are the limitations?"

Say:

"Auto transcription is still not perfect for every song and language. The system is strongest for English and major Indian languages, and it still needs stronger multi-pass scoring for truly global song transcription."

What You Should Understand Deeply

If you only memorize a few things, memorize these:

1. The transcript is the central data layer.
2. The timeline is updated through structured operations.
3. Captions are stored as text overlays and rendered through ASS/FFmpeg.
4. B-roll is generated from beats and visual meaning, not only raw keywords.
5. Preview and export use background jobs.
6. The project added language-aware fixes because generic English-first logic broke real multilingual behavior.

Viva Q and A

Q: Why did you use a timeline if the transcript already exists?

A: Because transcript editing is semantically powerful but not enough for final polish. The timeline gives manual control over placement, pacing, trim, and overlay adjustments.

Q: Why store timeline state in JSON?

A: It makes operations and versions easier to manage while keeping the system flexible for a prototype with many evolving feature types.

Q: Why is B-roll not fully automatic?

A: Because semantic visual interpretation is subjective and often weak for poetic or non-English text. Human override improves trust and usability.

Q: Why did subtitle rendering break for Kannada but not English?

A: English can often tolerate simpler rendering assumptions. Kannada requires proper complex-script shaping, so some English-style formatting tricks break glyph composition.

Q: Why is transcription quality so central?

A: Because transcript quality directly affects captions, text-based edits, and B-roll query generation. If transcript quality is weak, all downstream AI features degrade.

Q: What would you improve next if this became a product?

A: I would build stronger multi-pass auto transcription scoring, broader explicit language support, better B-roll reranking, and richer timeline controls.

How to Narrate the Demo

Use this story:

1. Create project and upload media.
2. Generate transcript and show that words are timed.
3. Make a text-based edit to prove transcript-driven workflow.
4. Add captions and explain the language-aware rendering path.
5. Open B-roll Studio and show original lyric, English gloss, and query reasoning.
6. Sync chosen B-roll to timeline.
7. Drag or trim B-roll manually.
8. Render preview and explain background jobs.

That sequence demonstrates the whole system clearly and logically.

Final Technical Position

ClipMind is best understood as an integrated AI editing system, not a single-model application. The project combines timed text, structured timeline operations, language-aware rendering, semantic B-roll planning, and async media processing into one coherent pipeline. That integration is the strongest evidence that this is a real engineering project rather than a simple interface over an external model.